Звіт про лабораторну роботу №11. Структурні шаблони проєктування (варіант 2, «Обласна лікарня»)

Nov 14, 2026 · @Таня

Мета та завдання

Мета роботи — навчитися за словесним описом предметної області розпізнавати ситуації, де об'єкти потрібно узгодити, розширити або приховати без зміни їхнього коду, і розв'язувати їх структурними шаблонами Adapter, Decorator, Facade та Proxy.

Предметна область варіанта 2 — «Обласна лікарня». Чотири проблемні ситуації розв'язано чотирма різними шаблонами, кожен використано рівно один раз. Рішення змодельовано мовою Java 21 без UI, бази даних і сторонніх бібліотек; поведінку показано консольним журналом у форматі `[час] [джерело] подія`.

Завдання	Ситуація	Шаблон	Виконавець
1	Перегляд медичних знімків	Proxy	Студент А
2	Підключення лабораторного аналізатора	Adapter	Студент А
3	Госпіталізація пацієнта	Facade	Студент Б
4	Позначки в картці пацієнта	Decorator	Студент Б

Прізвища студентів А і Б, групу й дату здачі потрібно вписати перед подачею звіту.

Ситуація 1. Перегляд медичних знімків — Proxy

Знімки КТ і МРТ великі й лежать у віддаленому сховищі, тож їх потрібно завантажувати лише при першому відкритті та показувати лише лікарям.

Проблема. Картка пацієнта завантажувала всі знімки одразу при відкритті, а переглядати їх могли навіть реєстратори.

Чому Proxy. Потрібен замісник із тим самим інтерфейсом, який керує доступом до дорогого об'єкта: відкладає його створення (віртуальний Proxy) і перевіряє роль працівника (захисний Proxy). Decorator не підходить, бо додає поведінку, а

не контролює доступ і життєвий цикл. Facade не підходить, бо об'єкт один і його інтерфейс не спрощується.

Реалізація (пакет `ua.hospital.imaging`):

Клас	Роль у шаблоні	Призначення
`MedicalImage`	Subject	Спільний інтерфейс: `id()`, `title()`, `open(viewer)`
`ImageProxy`	Proxy	Перевіряє роль, створює справжній знімок лише при першому відкритті
`RemoteMedicalImage`	RealSubject	Справжній знімок; дані завантажуються зі сховища при створенні
`PatientImageCard`	Client	Картка пацієнта; працює лише з `MedicalImage`
`ImageStorage`	Допоміжний	Віддалене сховище, лічильник завантажень

Перевірка ролі стоїть перед кешем: реєстратор отримує відмову, навіть коли знімок уже завантажено.

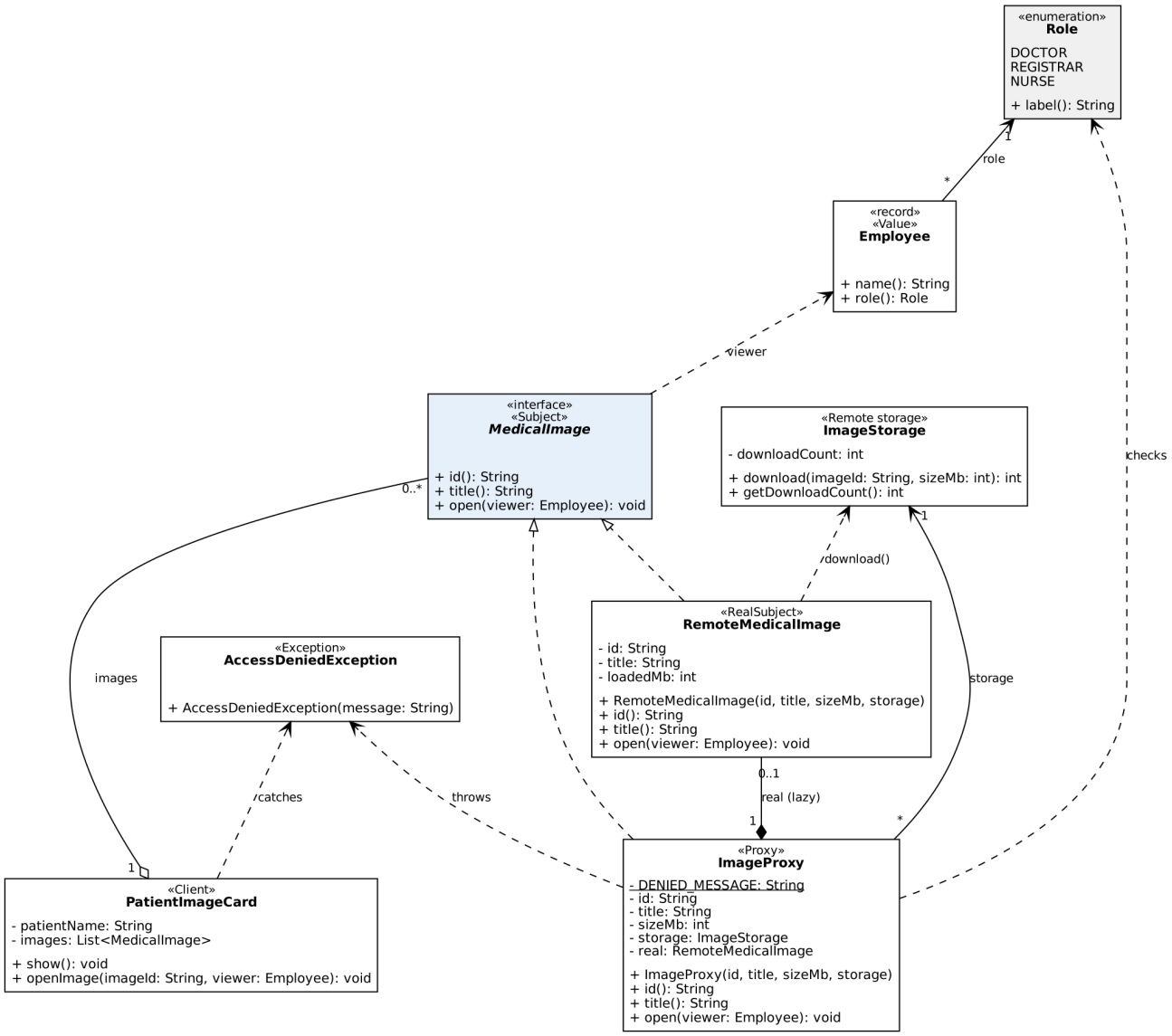
Фрагмент журналу.

```
[08:00:25] [Main] Завантажень зі сховища після відкриття картки: 0
[08:00:35] [ImageProxy[MRI-102]] Перше відкриття: створюю справжній знімок
[08:00:40] [ImageStorage] Завантаження знімка MRI-102 з віддаленого сховища
(610 МБ)...
[08:00:50] [RemoteMedicalImage[MRI-102]] Відображає знімок «МРТ головного
мозку» для Шевченко (лікар)
[08:01:00] [ImageProxy[MRI-102]] Знімок уже в пам'яті, завантаження не
потрібне
[08:01:10] [Main] Завантажень зі сховища після двох відкриттів МРТ: 1
[08:01:20] [ImageProxy[MRI-102]] Відмова: роль «реєстратор» не має доступу
[08:01:25] [PatientImageCard] Не вдалося відкрити MRI-102: «Доступ до знімків
мають лише лікарі»
```

У журналі видно, що перевірку й створення виконує `ImageProxy`, а відображення — `RemoteMedicalImage`.

Діаграма класів.

Завдання 1. Proxy — перегляд медичних знімків



Ситуація 2. Підключення лабораторного аналізатора — Adapter

Драйвер нового аналізатора повертає результати рядком із кодами показників у мкмоль/л, а система лікарні очікує об'єкти з назвами показників у ммоль/л; змінити драйвер не можна.

Чому Adapter. Інтерфейс драйвера несумісний із тим, що очікує система, і його неможливо змінити. Потрібно лише перекласти формат і назви. Facade не підходить: він спрощує набір підсистем із зручними інтерфейсами, а не узгоджує несумісний інтерфейс із наявним цільовим.

Реалізація (пакет `ua.hospital.lab`):

Клас	Роль у шаблоні	Призначення
`LabAnalyzer`	Target	Власний інтерфейс системи лікарні: `name()`, `fetchResults(patientId)`
`BioChemDriver`	Adaptee	Драйвер виробника, рядок `SMP=…;GLU=5600;…;UNIT=umol/L`
`BioChemDriverAdapter`	Adapter	Розбирає рядок, замінює коди назвами, мкмоль/л → ммоль/л
`ChemProLink`	Adaptee №2	Пристрій іншого виробника, записи `GLUCOSE@5900`
`ChemProLinkAdapter`	Adapter №2	Те саме для іншого формату
`HospitalLabSystem`	Client	Знає лише `LabAnalyzer` і `LabResult`
`Units`	Допоміжний	Перетворення одиниць

Розбір рядка й перетворення одиниць зосереджені в адаптері. Другий аналізатор підключено без жодної зміни в `HospitalLabSystem`.

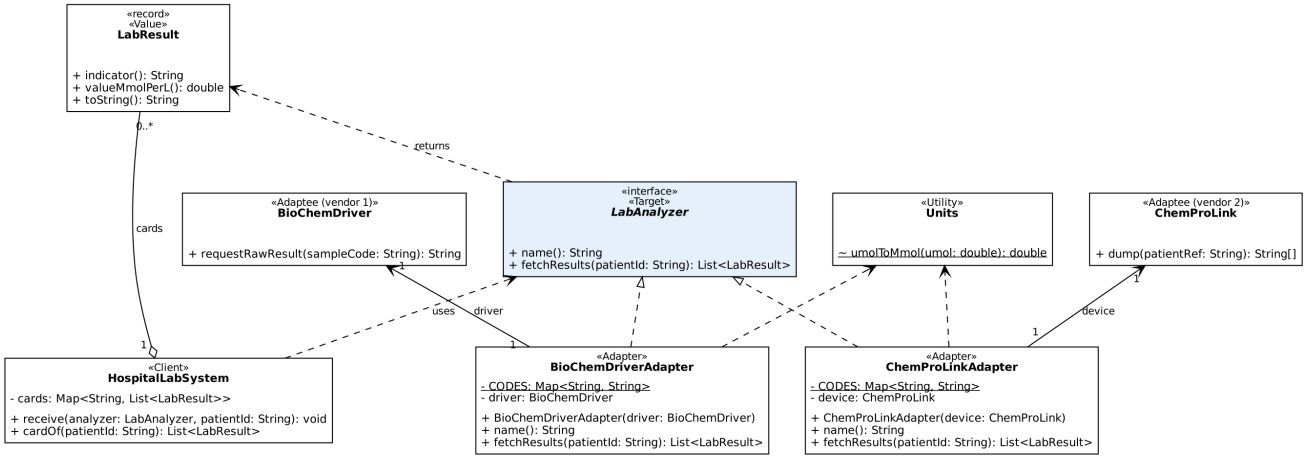
Фрагмент журналу.

```
[08:01:35] [BioChemDriver (виробник)] Повернув рядок: SMP=P-
001;GLU=5600;UREA=4200;CREA=88;UNIT=umol/L
[08:01:50] [BioChemDriverAdapter] Викликаю драйвер
BioChemDriver.requestRawResult(P-001)
[08:02:00] [BioChemDriverAdapter] Рядок розібрано й перетворено (мкмоль/л →
ммоль/л), результатів: 3
[08:02:05] [HospitalLabSystem] Записано у картку P-001: Глюкоза = 5.600
ммоль/л
[08:02:35] [ChemProLink (виробник 2)] Повернув записи для P-002:
GLUCOSE@5900, UREA@4100, CREATININE@95
[08:02:45] [HospitalLabSystem] Записано у картку P-002: Глюкоза = 5.900
ммоль/л
```

Сирий формат виробника видно лише у джерелах `BioChemDriver` та `ChemProLink`; система лікарні у журналі працює вже з перетвореними значеннями.

Діаграма класів.

Завдання 2. Adapter — підключення лабораторного аналізатора



Ситуація 3. Госпіталізація пацієнта — Facade

Госпіталізація потребує п'яти підсистем, а порядок їхніх викликів був продубльований у двох місцях, тому після зміни правил харчоблок перестав отримувати повідомлення про планових пацієнтів.

Чому Facade. Потрібна одна проста операція над кількома підсистемами, з послідовністю в одному місці. Інтерфейси підсистем сумісні, узгоджувати нічого не треба, тому Adapter не підходить. Підсистеми залишаються публічними й доступними окремо.

Реалізація (пакет `ua.hospital.admission`):

Клас	Роль у шаблоні	Призначення
`HospitalizationFacade`	Facade	Єдина операція `hospitalize(patient)` із повним порядком кроків
`PatientRegistration`	Підсистема	Реєстрація пацієнта
`BedRegistry`	Підсистема	Реєстр ліжок; `transfer` доступний напряму
`DoctorAssignment`	Підсистема	Призначення лікуючого лікаря
`ExaminationPlanner`	Підсистема	План обстеження
`KitchenNotifier`	Підсистема	Повідомлення харчоблоку

Клас	Роль у шаблоні	Призначення
`AdmissionDepartment`, `PlannedHospitalizationModule`	Client	Працюють лише з фасадом

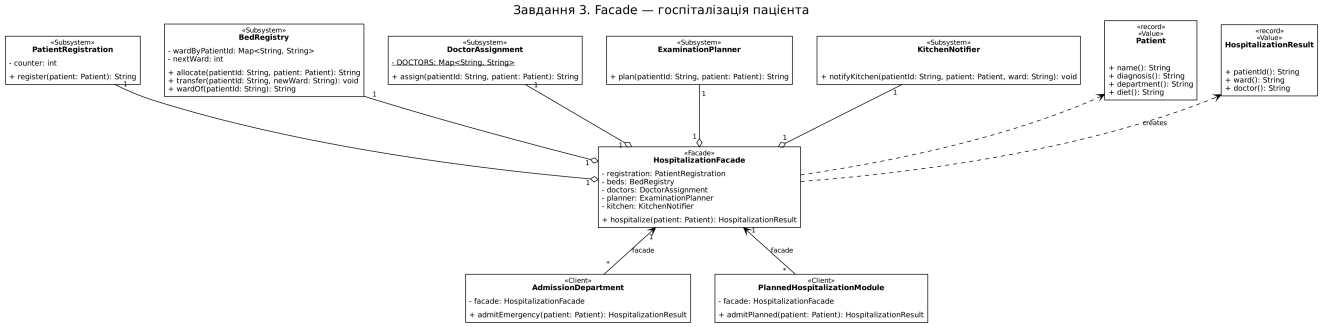
Порядок кроків: реєстрація, ліжко, лікар, план обстеження, харчоблок. Цей самий код викликають приймальне відділення й модуль планової госпіталізації, тому харчоблок отримує повідомлення в обох випадках.

Фрагмент журналу.

```
[08:03:05] [AdmissionDepartment] Екстрена госпіталізація: Гончар І. П.
[08:03:10] [HospitalizationFacade] Початок госпіталізації: Гончар І. П.
[08:03:15] [PatientRegistration] Зареєстровано пацієнта Гончар І. П., номер
P-1001
[08:03:20] [BedRegistry] Пацієнту P-1001 виділено ліжко: Палата 201
(Кардіологія)
[08:03:25] [DoctorAssignment] Пацієнту P-1001 призначено лікаря: д-р
Бондаренко
[08:03:35] [KitchenNotifier] Харчоблок отримав повідомлення: P-1001, Палата
201 (Кардіологія), дієта: стіл №10
[08:03:50] [PlannedHospitalizationModule] Планова госпіталізація: Лисенко Т.
В.
[08:04:20] [KitchenNotifier] Харчоблок отримав повідомлення: P-1002, Палата
202 (Неврологія), дієта: стіл №5
[08:04:35] [BedRegistry] Пацієнта P-1001 переведено: Палата 201 (Кардіологія)
→ Палата 305 (Кардіологія)
```

Останній запис показує окреме використання підсистеми без повторної госпіталізації.

Діаграма класів.



Ситуація 4. Позначки в картці пацієнта — Decorator

Позначки «алергія», «інфекційний контроль», «VIP» та «ризик падіння» поєднуються довільно й додаються під час лікування, тому окремий клас під кожну комбінацію дав надто багато майже однакових класів.

Чому Decorator. Позначки комбінуються під час виконання, а обгортка має той самий інтерфейс і додає поведінку. Proxy не підходить: він керує доступом до одного об'єкта, а декоратори нашаровуються й розширюють функціональність.

Реалізація (пакет `ua.hospital.card`):

Клас	Роль у шаблоні	Призначення
`PatientCard`	Component	Інтерфейс: `render()`, `assignProcedure(procedure)`
`BasicPatientCard`	ConcreteComponent	Звичайна картка без позначок
`CardMark`	Decorator	Абстрактна обгортка над будь-якою карткою
`AllergyMark`, `FallRiskMark`, `VipMark`	ConcreteDecorator	Додають попередження у відображення
`InfectionControlMark`	ConcreteDecorator	Змінює відображення й правила роботи: нагадує про засоби захисту при призначенні процедури

Нова позначка — це новий клас; існуючі класи не змінюються. Позначки стекуються в довільному порядку й кількості, бо самі є `PatientCard`.

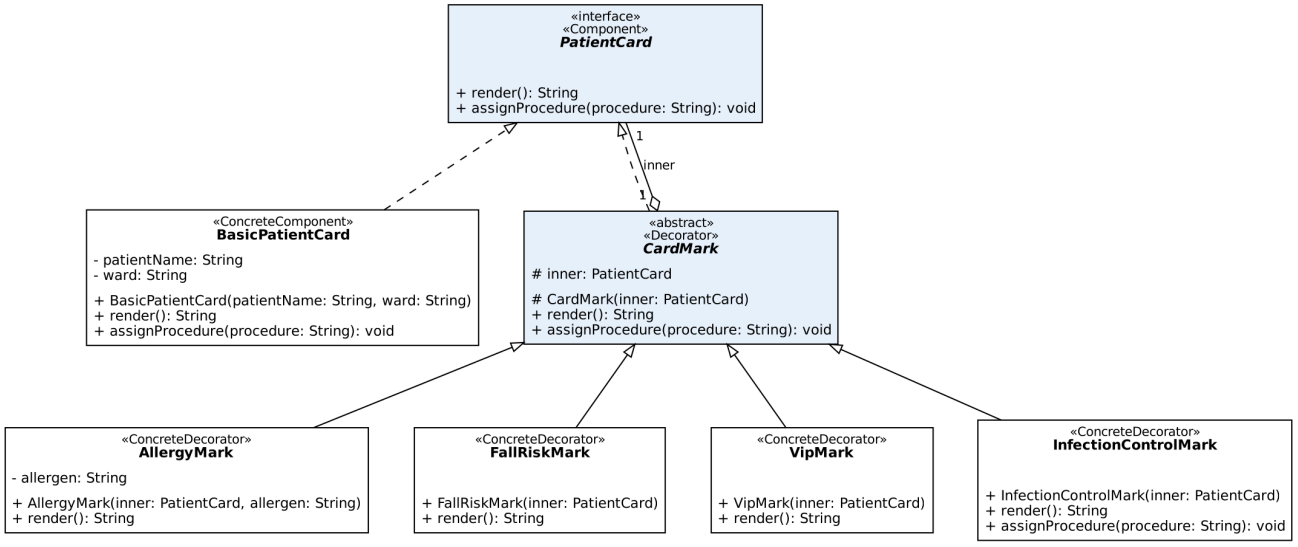
Фрагмент журналу.

```
[08:05:00] [FallRiskMark] Додає попередження про ризик падіння і передає
картку далі
[08:05:05] [AllergyMark] Додає попередження про алергію і передає картку далі
[08:05:10] [BasicPatientCard] Формує базове відображення картки
[08:05:15] [Main] Картка пацієнта: Ткачук Н. О., палата 210
    [АЛЕРГІЯ] Алерген: пеніцилін
    [РИЗИК ПАДІННЯ] Пересування лише в супроводі персоналу
[08:05:25] [InfectionControlMark] Нагадування: для процедури «внутрішньовенна
інфузія» використати засоби захисту (маска, рукавички, халат)
[08:05:30] [BasicPatientCard] Процедуру «внутрішньовенна інфузія» призначено
пацієнту Руденко В. М.
```

Ланцюжок викликів у журналі йде від зовнішньої обгортки до базової картки, і кожна ланка додає своє попередження.

Діаграма класів.

Завдання 4. Decorator — позначки в картці пацієнта



Порівняння шаблонів

Шаблон визначає ознака в описі ситуації: що саме потрібно зробити з об′єктом і чи можна змінити його інтерфейс.

Шаблон	Ознака в описі	Що робить	Чим відрізняється від сусідніх
Proxy	Дорогий об′єкт; завантаження лише при першому зверненні; перевірка прав	Той самий інтерфейс, керує доступом і створенням	Decorator додає поведінку, Proxy контролює доступ
Adapter	Інтерфейс, який змінити не можна, не збігається з потрібним	Перекладає формат, назви й одиниці	Facade спрощує групу підсистем, Adapter узгоджує несумісний інтерфейс
Facade	Одна послідовність викликів продубльована у кількох клієнтів	Одна проста операція над підсистемами	Підсистеми лишаються доступними, узгоджувати інтерфейси не треба

Шаблон	Ознака в описі	Що робить	Чим відрізняється від сусідніх
Decorator	Довільні комбінації розширень під час виконання, вибух кількості класів	Обгортка з тим самим інтерфейсом, що додає поведінку	Proxy керує одним об'єктом, декоратори нашаровуються

Крос-рев'ю та підготовка до захисту

Усі чотири ситуації пройшли попередню перевірку; живе взаємне рецензування студентів А і Б ще потрібно провести й зафіксувати.

Перевірки виконано автоматично: проєкт скомпільовано й запущено, журнал звірено з демонстраційними сценаріями, діаграми звірено з кодом.

Перевірка	Proxy	Adapter	Facade	Decorator
Діаграма збігається з кодом	так	так	так	так
Рішення усуває проблему з опису	так	так	так	так
Раннер відтворює демонстраційний сценарій	так	так	так	так
У журналі видно, хто діє: обгортка чи обгорнутий	так	так	так	так

Для захисту підготовлено аргументацію вибору кожного шаблону й типові питання (файл `docs/REVIEW.md` у проєкті). Обидва студенти мають уміти пояснити всі чотири рішення, незалежно від розподілу завдань.

Висновки

У ході роботи чотири проблемні ситуації лікарні розв'язано чотирма структурними шаблонами, кожен із яких застосовано рівно один раз.

- Proxy дозволив завантажувати великі знімки лише при першому відкритті (1 завантаження замість 4) і закрити доступ реєстраторам без змін у картці пацієнта.
- Adapter зосередив розбір рядка й перетворення мкмоль/л → ммоль/л в одному класі, а другий аналізатор підключено без змін у системі лікарні.

- Facade зібрав п’ять кроків госпіталізації в одну операцію, тому порядок кроків більше не дублюється в приймальному відділенні та модулі планової госпіталізації.
- Decorator замінив комбінаторне зростання класів картки обгортками, які додаються під час виконання без змін в існуючих класах.

Головна практична навичка — розрізняти шаблони за ознакою в описі: дорогий об’єкт і права доступу вказують на Proxy, несумісний інтерфейс на Adapter, продубльована послідовність викликів на Facade, довільні комбінації розширень на Decorator.